Software Design Description

miniaudioengine C++ SDK

Aidan Clyens

Contents

1. Introduction **3**

2. Stakeholders and Design Concerns **4**
2.1. Stakeholders 4
2.2. Design Concerns 4

3. Design Views **6**
3.1. Context View 6
3.2. Composition View 7
3.8.1. SDK Components 7
3.3. Logical View 9
3.3.1. Record Audio/MIDI Input 9
3.3.2. Play Audio/MIDI File 9
3.3.3. Playback to Output Device 10
3.3.4. Audio/MIDI Processing 10
3.3.5. Multiple Tracks 11
3.4. Dependency View 12
3.4.1. Layer Hierarchy 13
3.4.2. External Library Adapters 14
3.5. Information View 15
3.5.1. Audio Session 15
3.5.2. Device Service 16
3.5.3. File Service 17
3.5.4. Track Service 18
3.6. Interface View 19
3.6.1. Public SDK 19
3.7. Interaction View 21
3.7.1. Control Plane 21
3.7.2. Data Plane 23
3.8. Structure View 25
3.8.1. Library Structure 25
3.8.2. Project Structure 27
3.8.3. Thread Structure 28

4. Design Rationale **30**
4.1. Architectural Design 30
4.1.1. Audio Session 30

4.1.2. Services 32
4.1.3. Adapters 32
4.1.4. Handles 32
4.1.3. Software Design Patterns 32
4.2. External Libraries 34

1. Introduction

2. Stakeholders and Design Concerns

2.1. Stakeholders

Stakeholder	Responsibilities	Design Concerns
Software User	- Monitor audio input device. - Monitor MIDI input device. - Read audio files - Play audio to output device. - Play MIDI to output device.	DC-01 DC-02 DC-03 DC-04 DC-05 DC-06
Third-Party Developer	- Include SDK in C++ software application. - Handle incoming MIDI messages. - Process audio input. - Route processed audio to output device. - Manage audio in multiple tracks.	DC-07 DC-08 DC-09 DC-10 DC-11 DC-12 DC-13 DC-14 DC-17 DC-18
Maintainer	- Maintain CI/CD pipeline. - Manage code repo. - Manage software releases.	DC-13 DC-14 DC-15 DC-16 DC-17
Hardware	- Run on a Windows desktop. - Run on an embedded Linux platform.	DC-13 DC-14

2.2. Design Concerns

ID	Description	Relevant Views
DC-01	Monitor audio input device.	Logical, Interface, Interaction
DC-02	Monitor MIDI input device.	Logical, Interface, Interaction
DC-03	Open and read WAV audio files.	Logical, Information, Interaction

ID	Description	Relevant Views
DC-04	Open and read MIDI files.	Logical, Information
DC-05	Route audio to output device.	Logical, Interface, Interaction
DC-06	Route MIDI to output device.	Logical, Interface
DC-07	Processing incoming MIDI messages.	Logical, Interface
DC-08	Processing incoming audio streams.	Logical, Information, Interface
DC-09	Manage multiple audio tracks.	Logical, Information, Interface, Interaction
DC-10	Add one audio or MIDI input to a track.	Logical, Information, Interface
DC-11	Attach one audio or MIDI output to a track.	Logical, Information, Interface
DC-12	Chain multiple audio processors in one track.	Logical, Information, Interface
DC-13	Build software SDK on Windows and Linux.	Context, Structure
DC-14	Build software SDK on x86_64 and ARM64 platforms.	Context, Structure
DC-15	CI/CD pipeline builds and packages software on all compatible platforms.	Structure
DC-16	Complete unit testing and code coverage.	Structure
DC-17	Package software as an SDK used by third-party software.	Context, Composition, Structure
DC-18	Third-party software developers manage audio tracks, system audio/MIDI devices, and filesystem.	Composition, Interface, Interaction

3. Design Views

3.1. Context View

Describes the software in context with its external environment. Define users, external components, and the interactions between.

Design Concern	
DC-13	Build software SDK on Windows and Linux.
DC-14	Build software SDK on x86_64 and ARM64 platforms.
DC-17	Package software as an SDK used by third-party software.

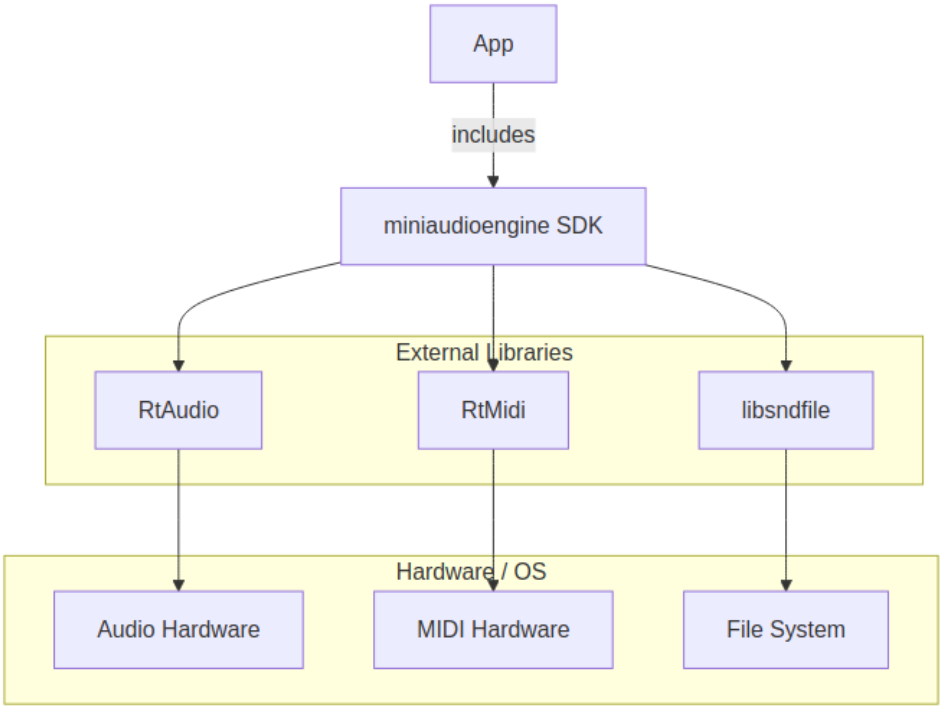

Figure 1: miniaudioengine SDK environment context

3.2. Composition View

3.8.1. SDK Components

Describe the composition of the **miniaudioengine** SDK software libraries.

Design Concern	
DC-17	Package software as an SDK used by third-party software.
DC-18	Third-party software developers manage audio tracks, system audio/MIDI devices, and filesystem.

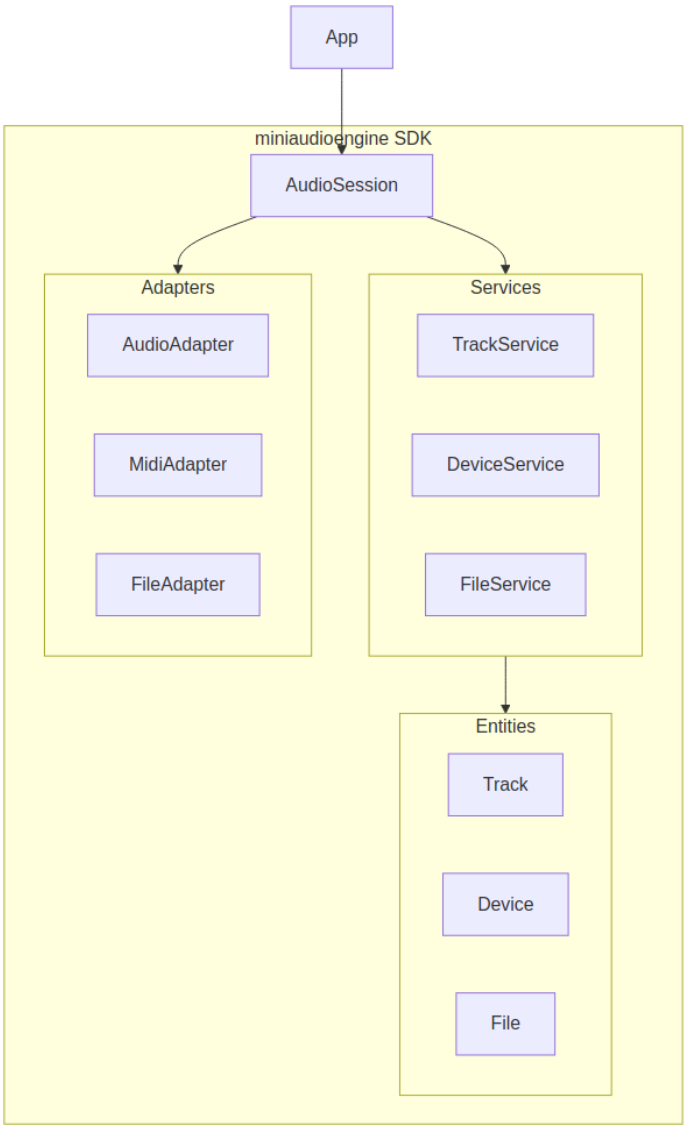

Figure 2: miniaudioengine SDK composition

3.3. Logical View

3.3.1. Record Audio/MIDI Input

Design Concern	
DC-01	Monitor audio input device.
DC-02	Monitor MIDI input device.

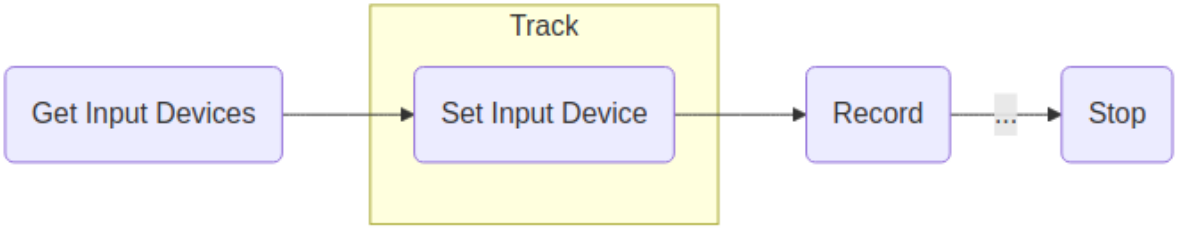

Figure 3: Diagram 3

3.3.2. Play Audio/MIDI File

Design Concern	
DC-03	Open and read WAV audio files.
DC-04	Open and read MIDI files.

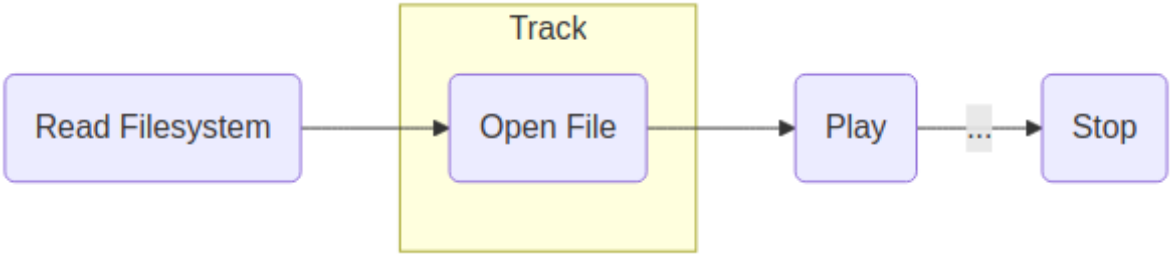

Figure 4: Diagram 4

3.3.3. Playback to Output Device

Design Concern	
DC-05	Route audio to output device.
DC-06	Route MIDI to output device.

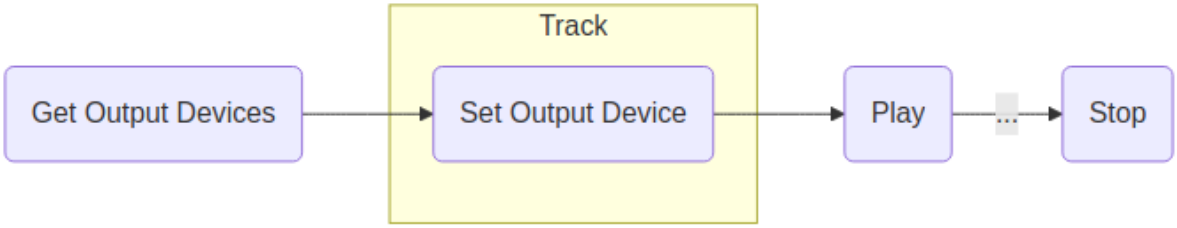

Figure 5: Diagram 5

3.3.4. Audio/MIDI Processing

Design Concern	
DC-07	Processing incoming MIDI messages.
DC-08	Processing incoming audio streams.

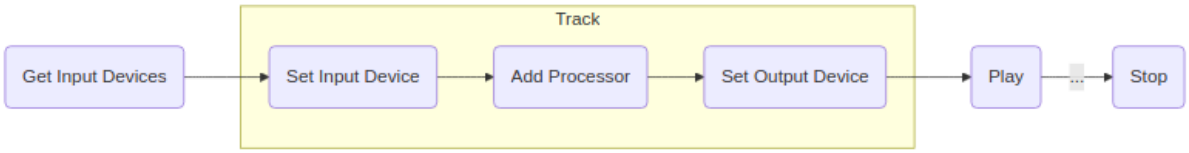

Figure 6: Diagram 6

3.3.5. Multiple Tracks

Design Concern	
DC-09	Manage multiple audio tracks.
DC-10	Add one audio or MIDI input to a track.
DC-11	Attach one audio or MIDI output to a track.
DC-12	Chain multiple audio processors in one track.

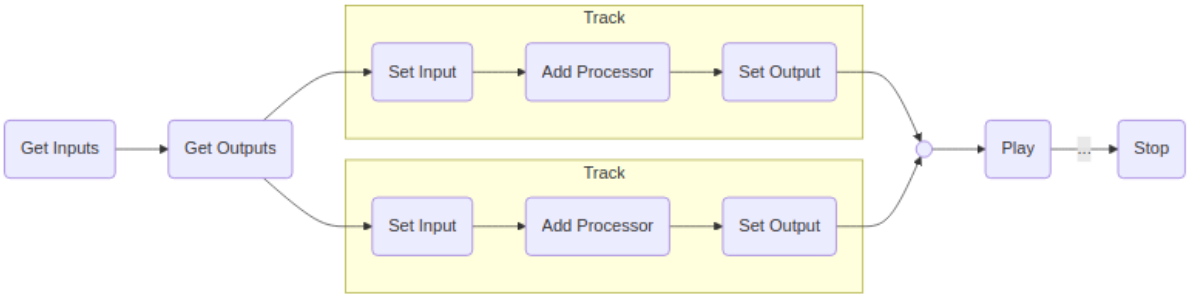

Figure 7: Diagram 7

3.4. Dependency View

The components in this SDK depend on the file system, audio and MIDI devices on the host system. Separating devices and files into different services divides the dependency.

3.4.1. Layer Hierarchy

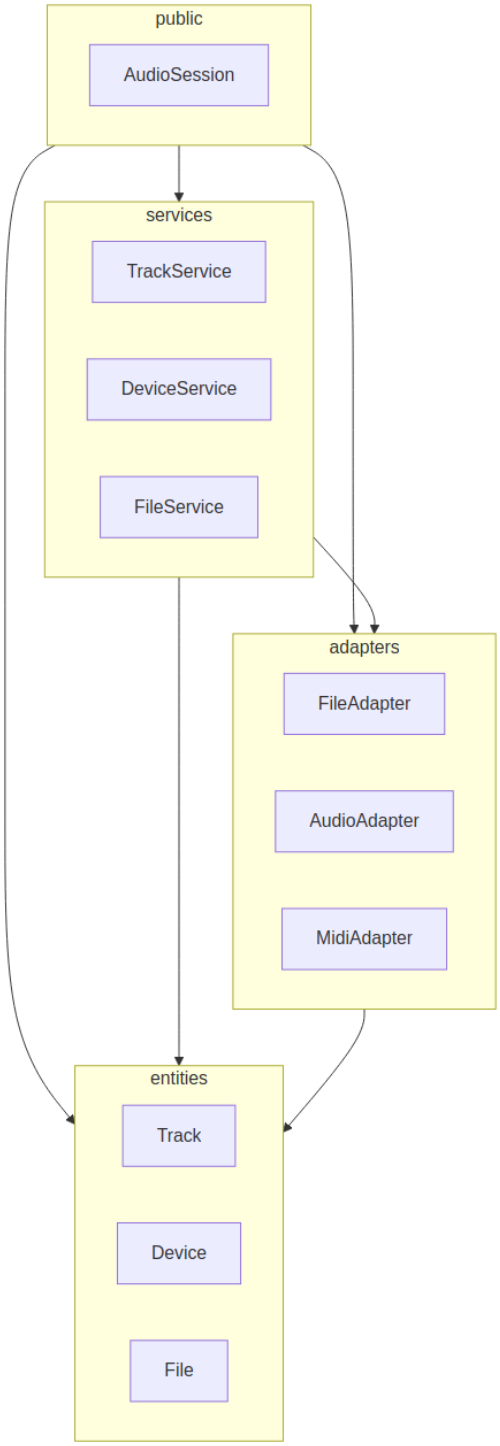

Figure 8: Diagram 8

3.4.2. External Library Adapters

3.5. Information View

Design Concern	
DC-03	Open and read WAV audio files.
DC-04	Open and read MIDI files.
DC-08	Processing incoming audio streams.
DC-09	Manage multiple audio tracks.
DC-10	Add one audio or MIDI input to a track.
DC-11	Attach one audio or MIDI output to a track.
DC-12	Chain multiple audio processors in one track.

3.5.1. Audio Session

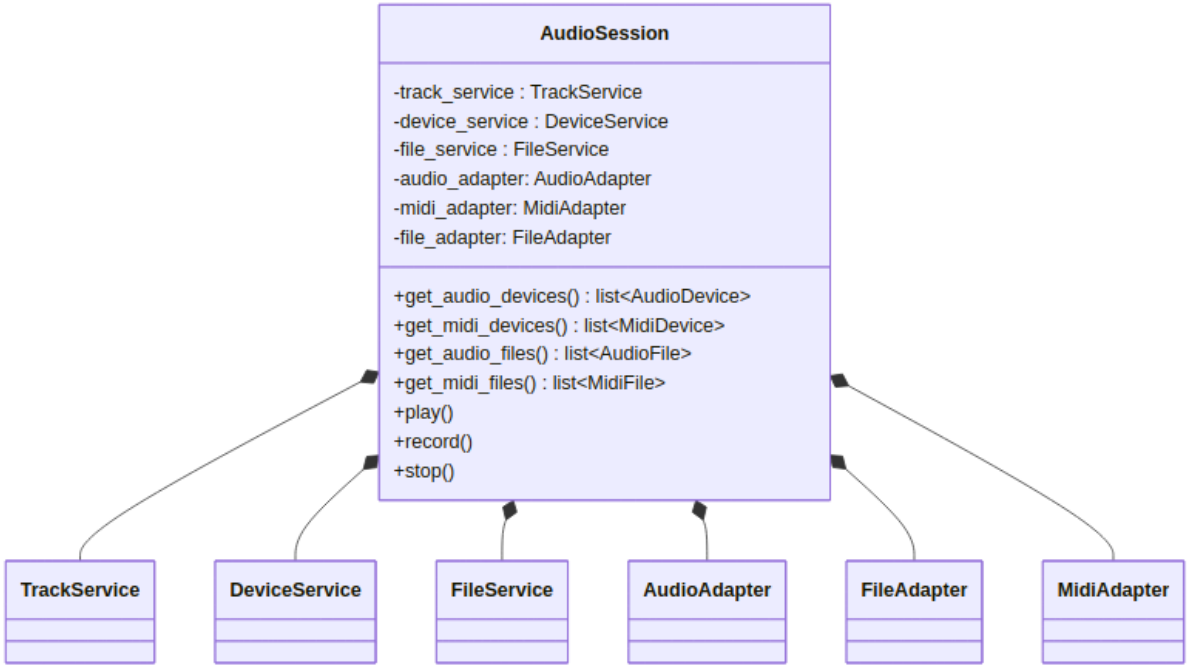

Figure 9: Diagram 9

3.5.2. Device Service

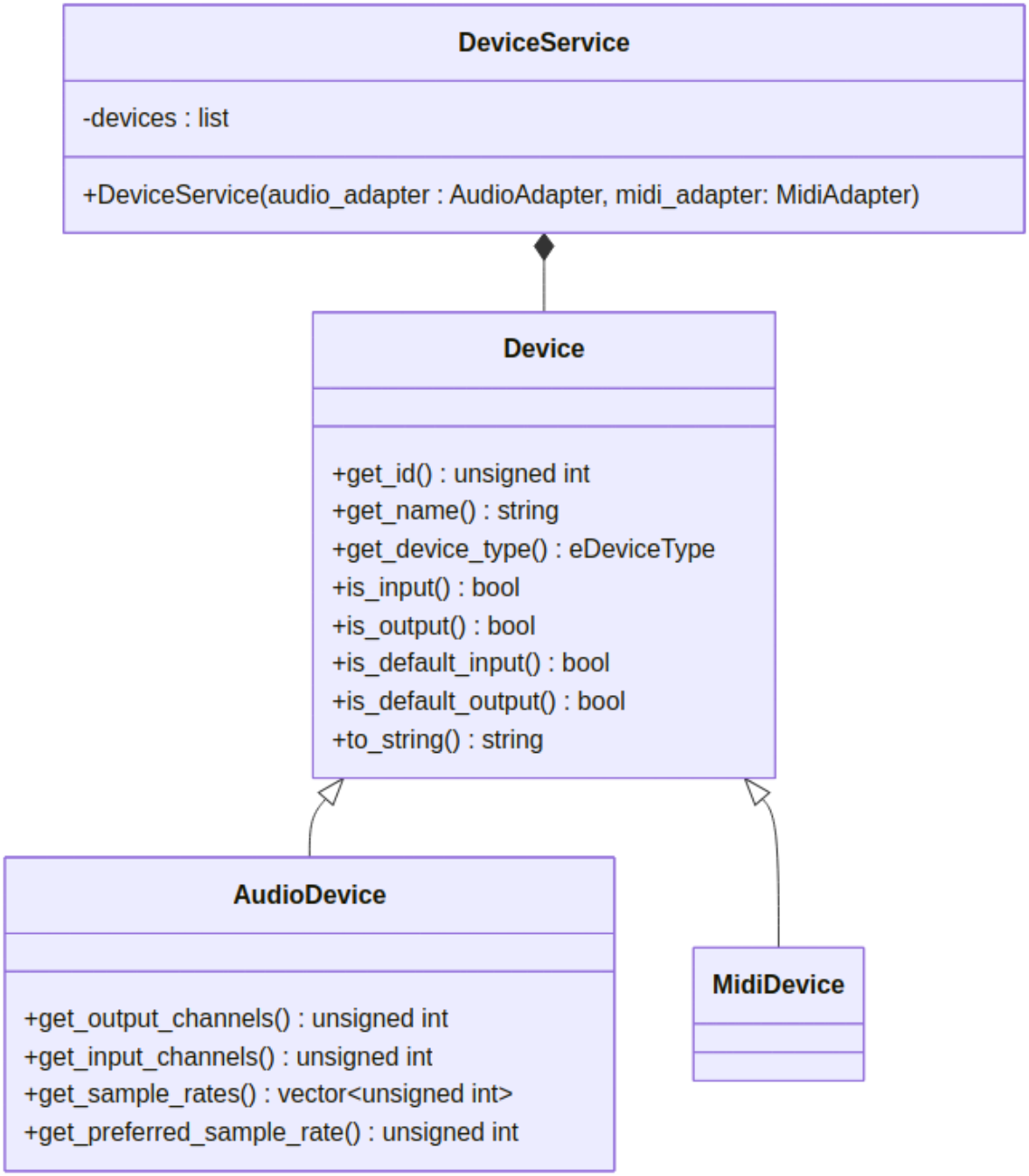

Figure 10: Diagram 10

3.5.3. File Service

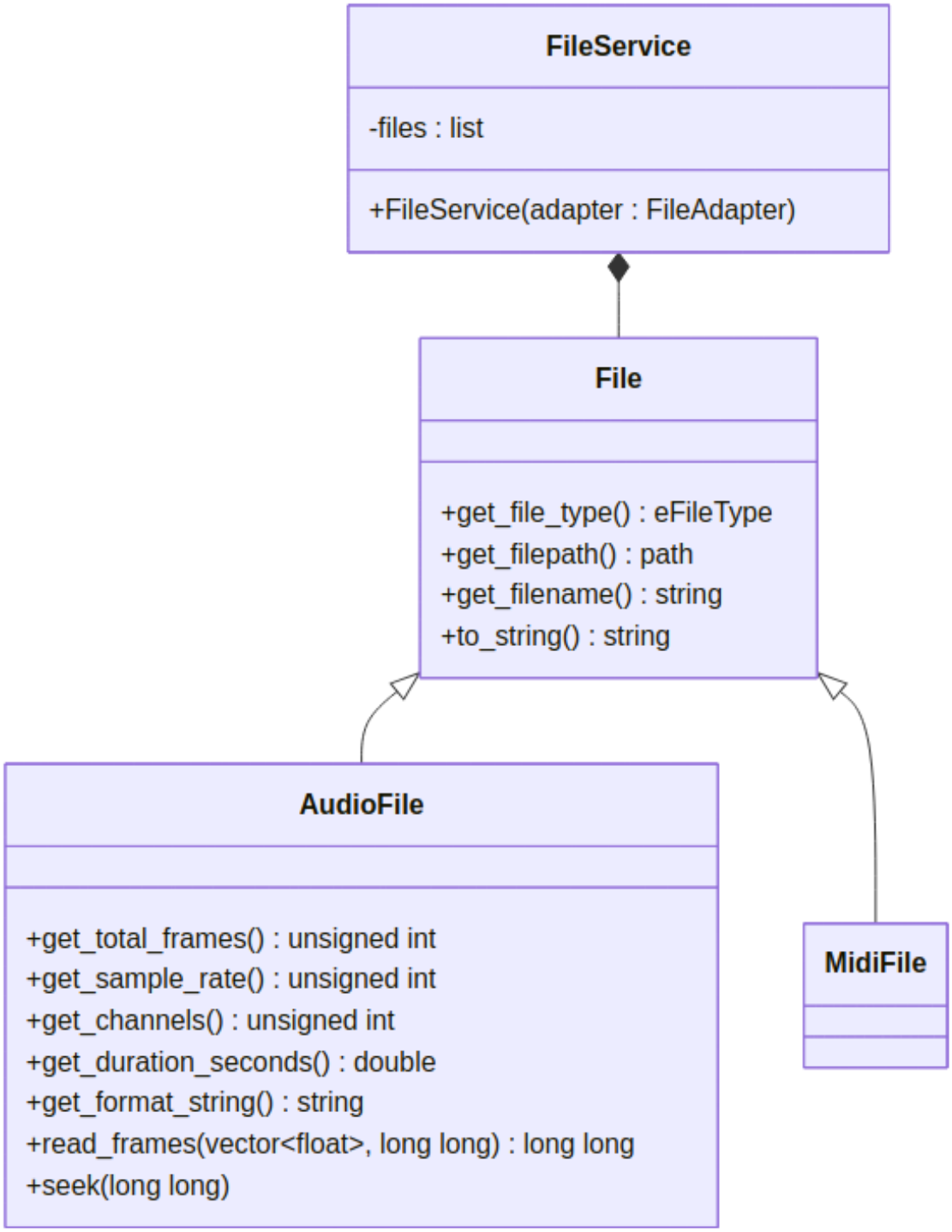

Figure 11: Diagram 11

3.5.4. Track Service

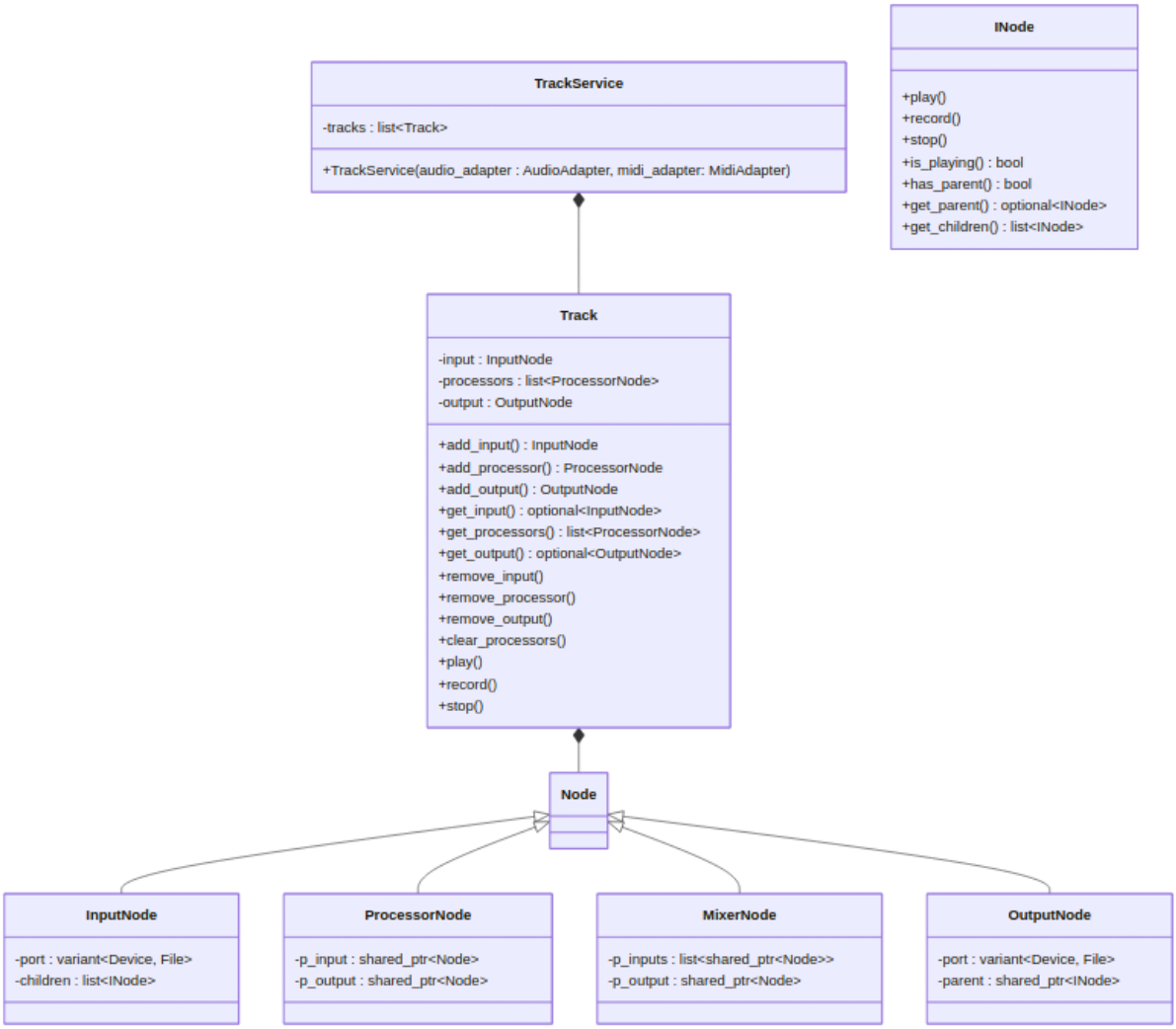

Figure 12: Diagram 12

3.6. Interface View

3.6.1. Public SDK

Design Concern	
DC-01	Monitor audio input device.
DC-02	Monitor MIDI input device.
DC-05	Route audio to output device.
DC-06	Route MIDI to output device.
DC-07	Processing incoming MIDI messages.
DC-08	Processing incoming audio streams.
DC-09	Manage multiple audio tracks.
DC-10	Add one audio or MIDI input to a track.
DC-11	Attach one audio or MIDI output to a track.
DC-12	Chain multiple audio processors in one track.
DC-18	Third-party software developers manage audio tracks, system audio/MIDI devices, and filesystem.

The user uses the software via the main `miniaudioengine` SDK library.

The following types need to be accessible to the user:

- `AudioSession`
- `Device`
- `File`
- `Track`
- `InputNode`
- `OutputNode`
- `ProcessorNode`

The following operations need to be accessible to the user:

Operation	Component
Get audio/MIDI devices	`AudioSession`, `Device`
Get audio/MIDI files	`AudioSession`, `File`
Set audio/MIDI device as input or output	`Track`, `InputNode`, `OutputNode`

Operation	Component
Set audio/MIDI file as input or output	`Track`, `InputNode`, `OutputNode`
Add a new track	`AudioSession`, `Track`
Add and audio or MIDI processor to a track	`Track`, `ProcessorNode`
Start/stop playback	`AudioSession`, `Track`
Start/stop recording	`AudioSession`, `Track`
Start/stop monitoring	`AudioSession`, `Track`

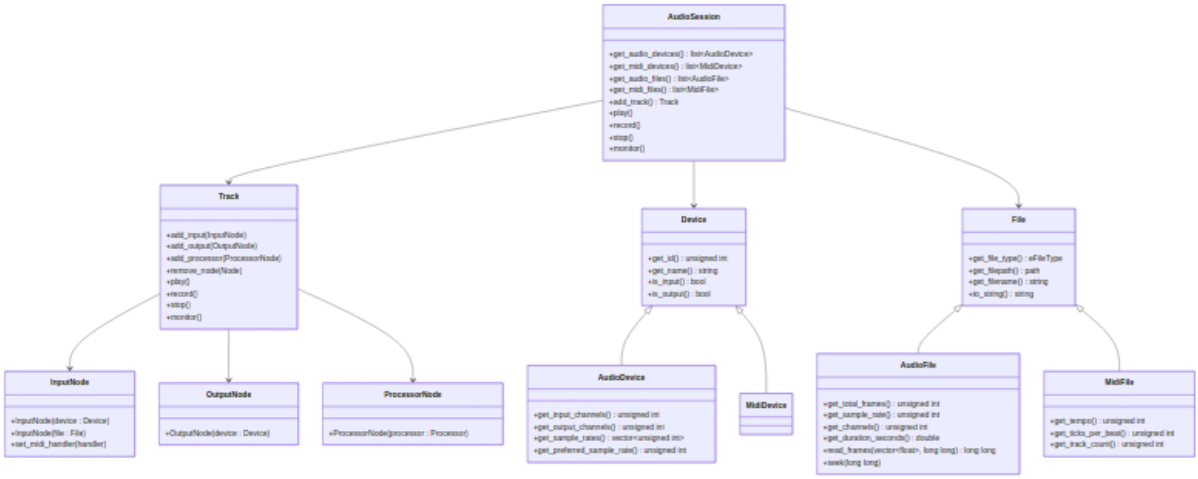

Figure 13: Diagram 13

3.7. Interaction View

Design Concern	
DC-03	Open and read WAV audio files.
DC-05	Route audio to output device.
DC-09	Manage multiple audio tracks.
DC-18	Third-party software developers manage audio tracks, system audio/MIDI devices, and filesystem.

3.7.1. Control Plane

- Get audio/MIDI devices
- Get audio/MIDI files
- Set audio/MIDI device as input or output
- Set audio/MIDI file as input or output
- Add a new track
- Add and audio or MIDI processor to a track
- Start/stop playback
- Start/stop recording

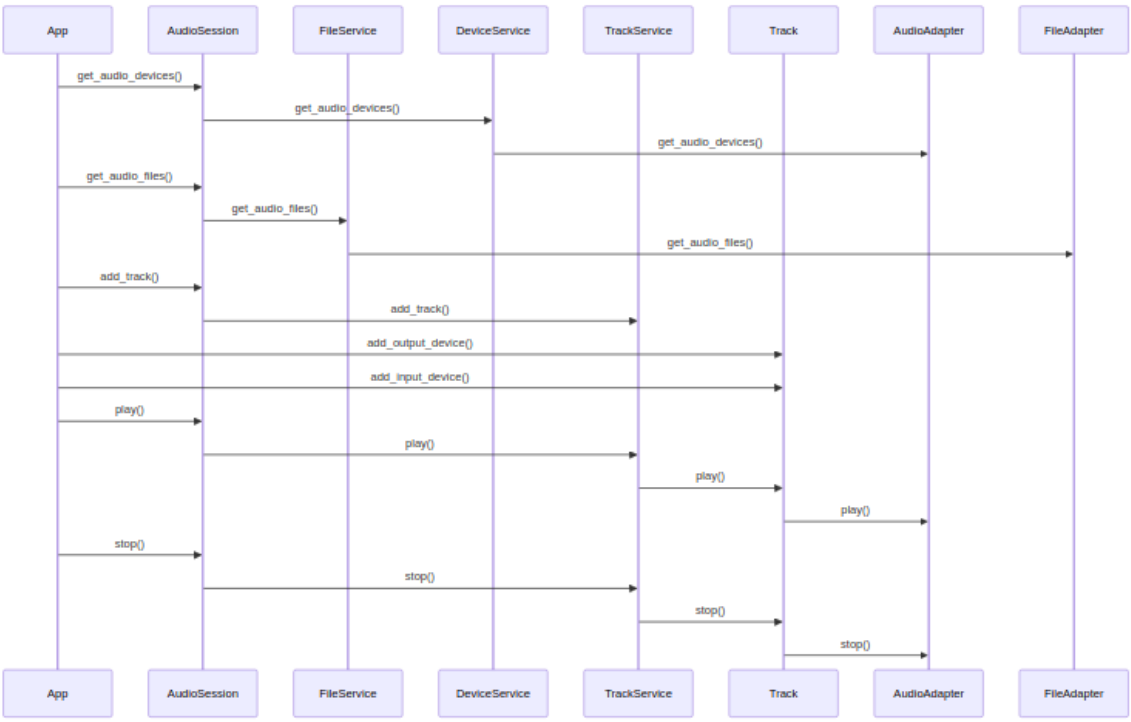

Figure 14: Diagram 14

3.7.2. Data Plane

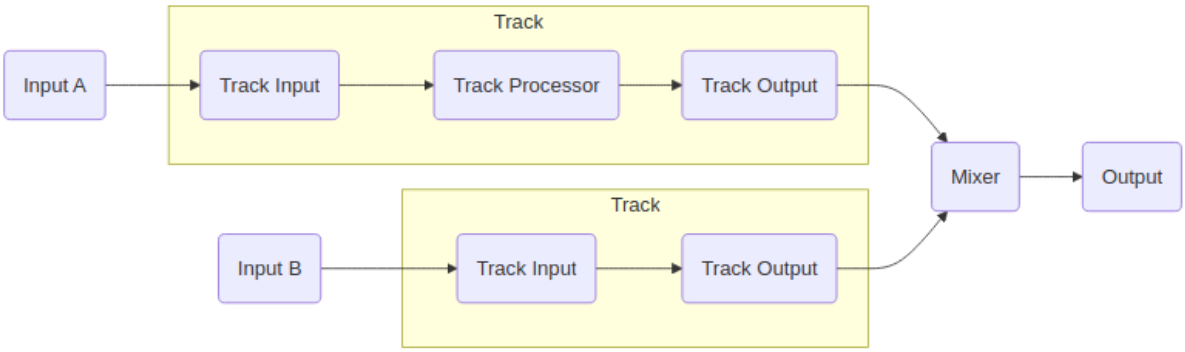

Figure 15: Diagram 15

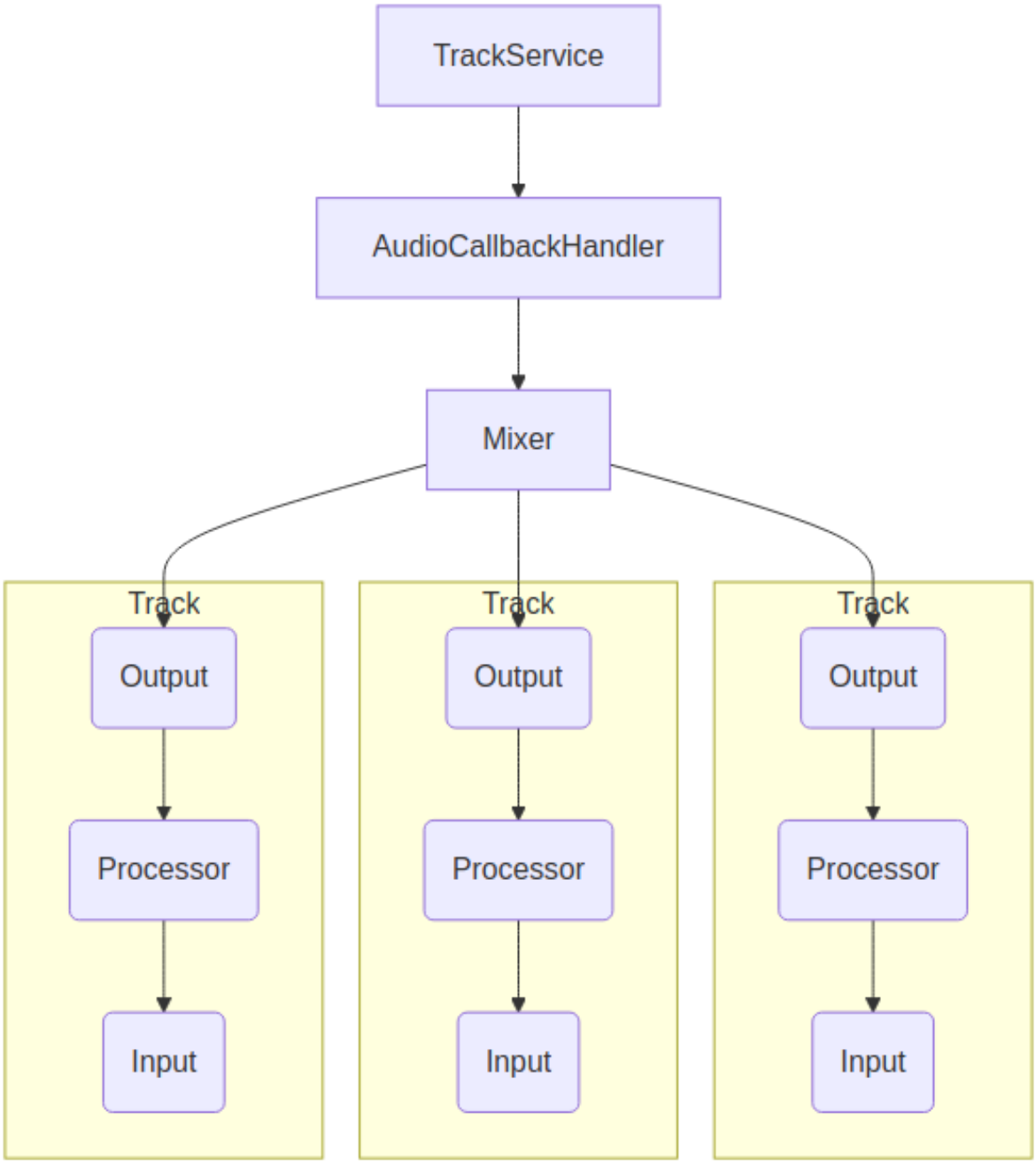

Figure 16: Diagram 16

3.8. Structure View

Design Concern	
DC-13	Build software SDK on Windows and Linux.
DC-14	Build software SDK on x86_64 and ARM64 platforms.
DC-15	CI/CD pipeline builds and packages software on all compatible platforms.
DC-16	Complete unit testing and code coverage.
DC-17	Package software as an SDK used by third-party software.

3.8.1. Library Structure

Internal Library	Components
miniaudioengine	
services	
adapters	
entities	
framework	

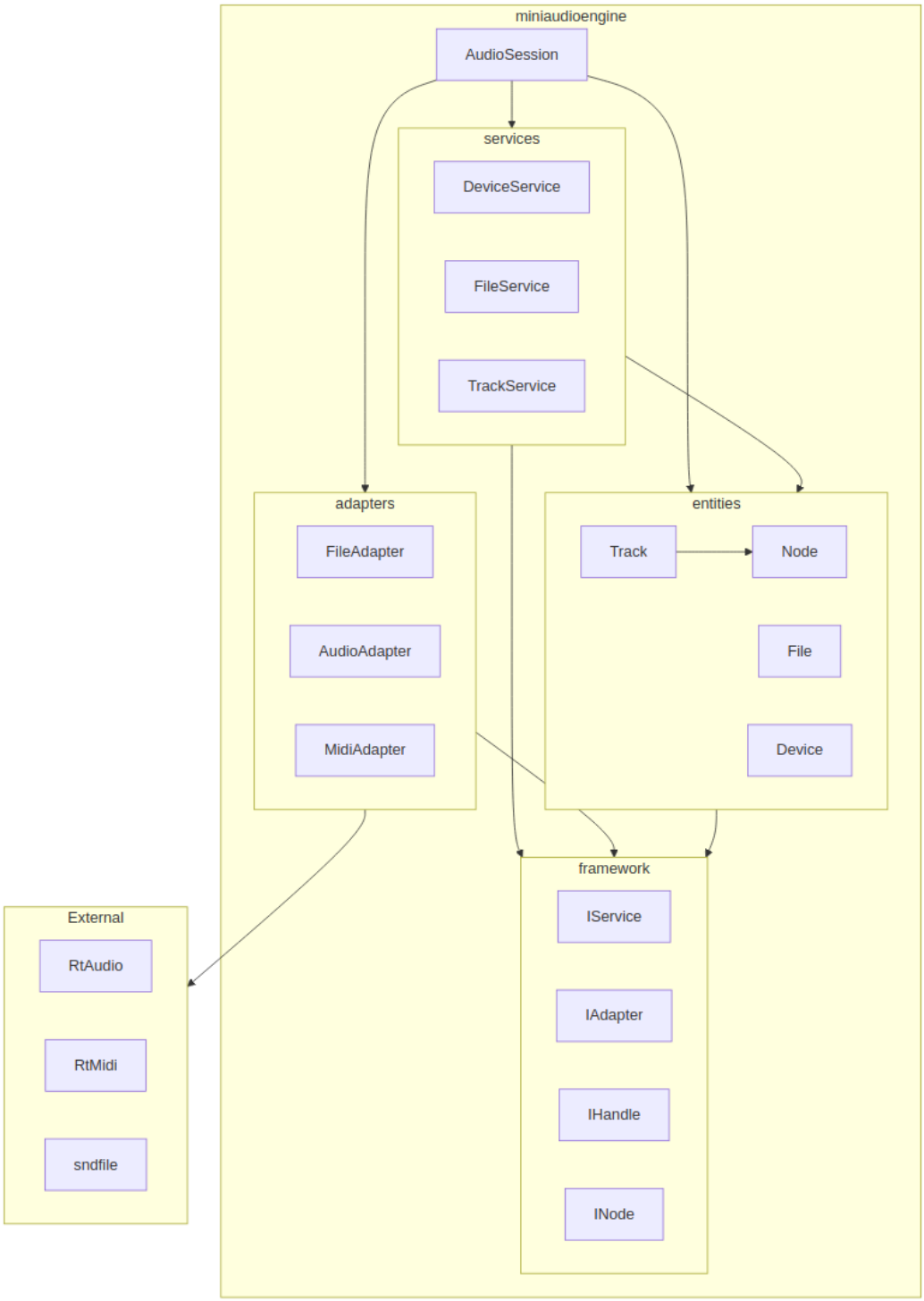

Figure 17: Diagram 17

3.8.2. Project Structure

```
examples/           # Example programs using miniaudioengine SDK
include/
    miniaudioengine/
        audiosession.h          # using namespace miniaudioengine
        track/
            track.h             # using namespace miniaudioengine
            input.h             # using namespace miniaudioengine
            output.h            # using namespace miniaudioengine
            processor.h         # using namespace miniaudioengine
            mixer.h             # using namespace miniaudioengine
        device/
            audiodevice.h       # using namespace miniaudioengine
            mididevice.h        # using namespace miniaudioengine
        file/
            audiofile.h         # using namespace miniaudioengine
            midifile.h          # using namespace miniaudioengine
src/
    framework/
        include/
          interfaces/
              service.h         # using namespace miniaudioengine::interfaces
              adapter.h         # using namespace miniaudioengine::interfaces
              handle.h          # using namespace miniaudioengine::interfaces
              device.h          # using namespace miniaudioengine::interfaces
              file.h            # using namespace miniaudioengine::interfaces
              node.h            # using namespace miniaudioengine::interfaces
        src/
            audiosession.cpp
    services/
        track/
            include/
                trackservice.h          # using namespace miniaudioengine::services
            src/
                trackservice.cpp        # using namespace miniaudioengine::services
        device/
            include/
                deviceservice.h         # using namespace miniaudioengine::services
            src/
                deviceservice.cpp       # using namespace miniaudioengine::services
        file/
            include/
```

```
                fileservice.h           # using namespace miniaudioengine::services
            src/
                fileservice.cpp         # using namespace miniaudioengine::services
    entities/
        include/
        src/
            audiodevice.cpp
            mididevice.cpp
            audiofile.cpp
            midifile.cpp
            track.cpp
            input.cpp
            output.cpp
            processor.cpp
            mixer.cpp
    adapters/
        include/
            audioadapter.h              # using namespace miniaudioengine::adapters
            midiadapter.h               # using namespace miniaudioengine::adapters
            fileadapter.h               # using namespace miniaudioengine::adapters
        src/
            audioadapter.cpp            # using namespace miniaudioengine::adapters
            midiadapter.cpp             # using namespace miniaudioengine::adapters
            fileadapter.cpp             # using namespace miniaudioengine::adapters
tests/
```

3.8.3. Thread Structure

Thread	Description
Main	Main execution thread.
RtAudio	Real-time audio data processing.
RtMidi	Real-time MIDI message handling.

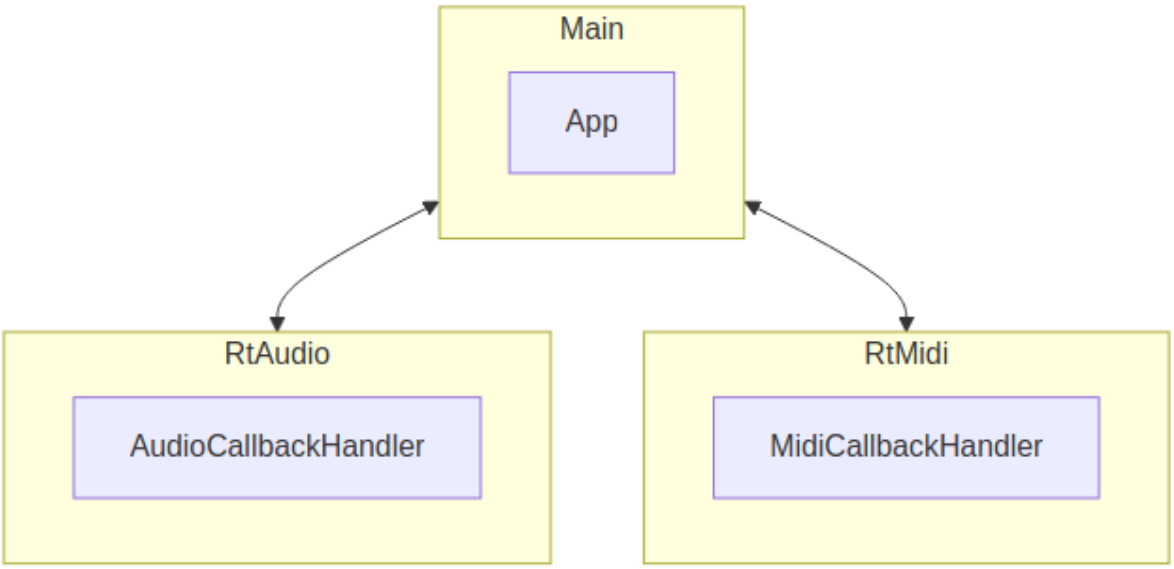

Figure 18: Diagram 18

4. Design Rationale

4.1. Architectural Design

I am keeping the control and data components of this library separate. The main component of the control section is the `AudioSession`. The `AudioSession` owns `Services` for different functionalities. `AudioSession` wraps function calls to the internal `Services` and exposes them to the programmer. `Services` own `Adapters` to interface with external hardware / libraries. `Services` return `Handle` objects for the programmer to interact with.

4.1.1. Audio Session

This SDK uses the Facade pattern for the `AudioSession` object. `AudioSession` is a wrapper around the internal logic exposed to the programmer. An `AudioSession` owns a `DeviceService`, `FileService`, and `TrackService` that divides and implements the internal engine logic.

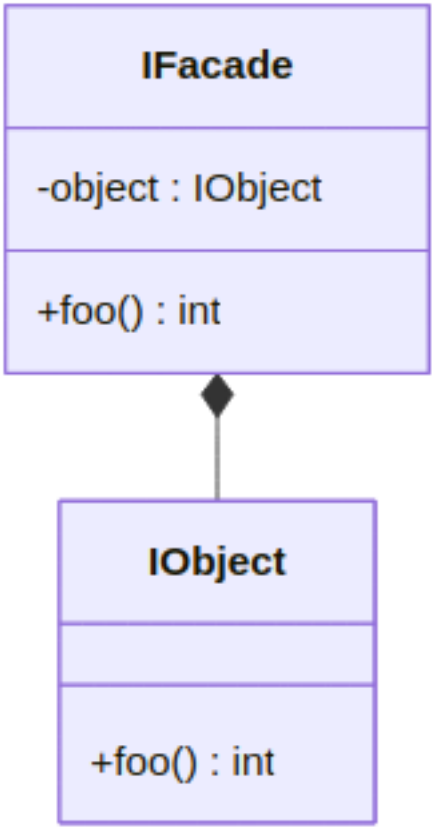

Figure 19: Diagram 19

where

```
int IFacade::foo() {
    return object.foo();
}
```

e.g.

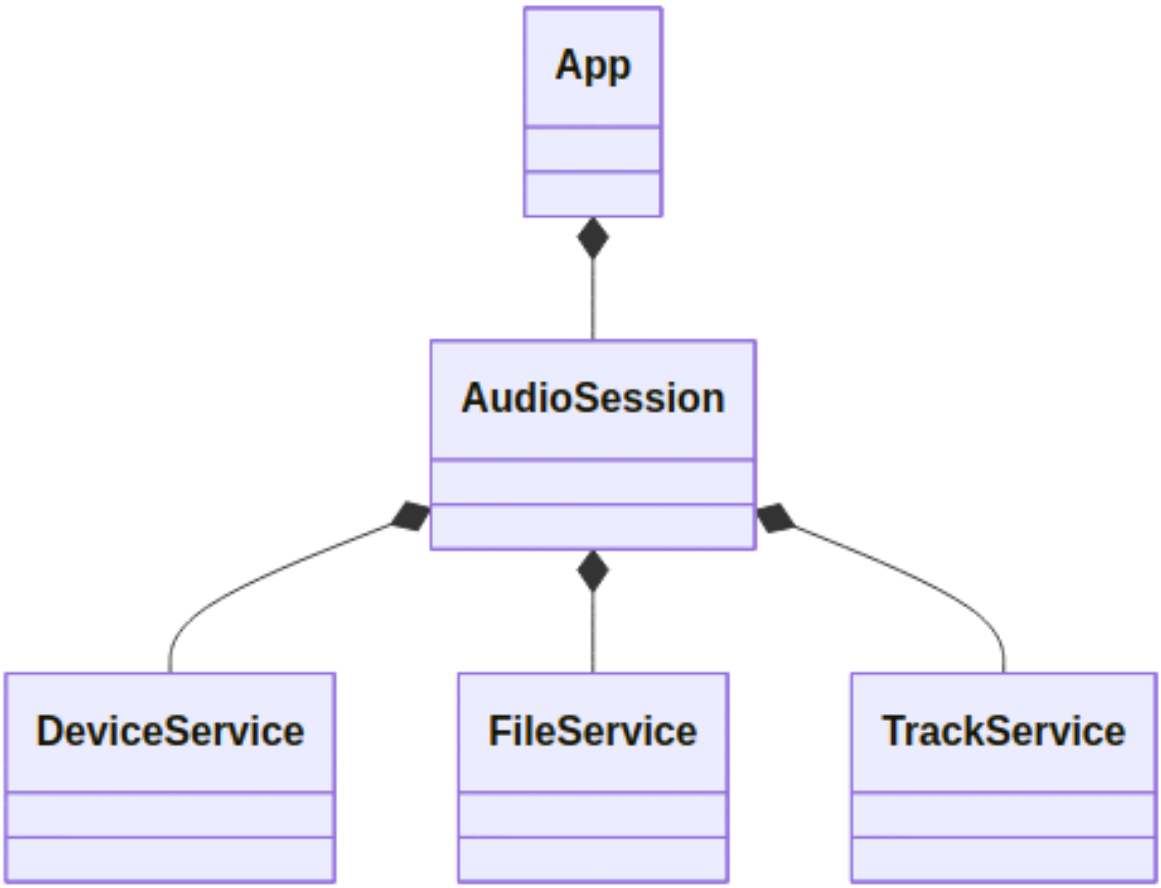

Figure 20: Diagram 20

4.1.2. Services

4.1.3. Adapters

This library uses the Adapter pattern to interface with external hardware / libraries. This adds an abstraction layer between the library and the external dependendies, providing isolation of implementation.

4.1.4. Handles

4.1.3. Software Design Patterns

C++ PImpl

Adapter

Factory

This SDK uses the Factory pattern to create `Device`, `File`, and `Node` objects.

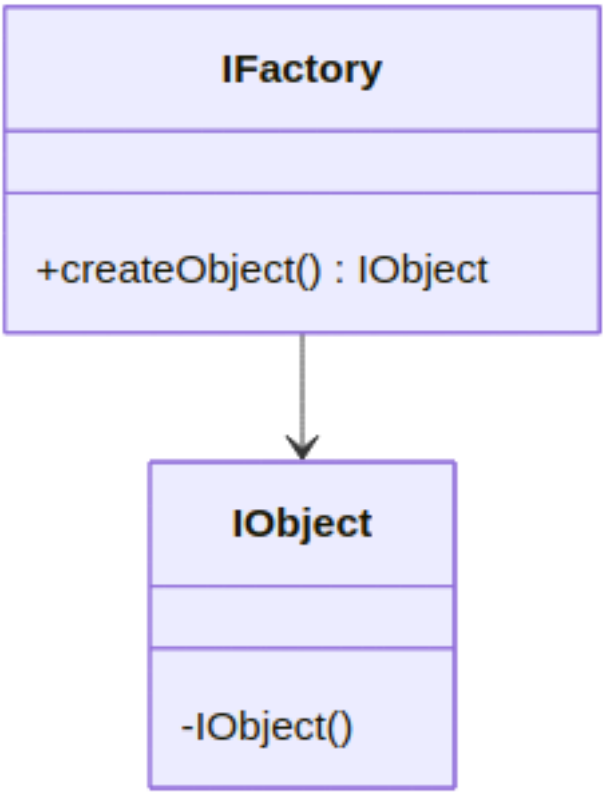

Figure 21: Diagram 21

e.g.

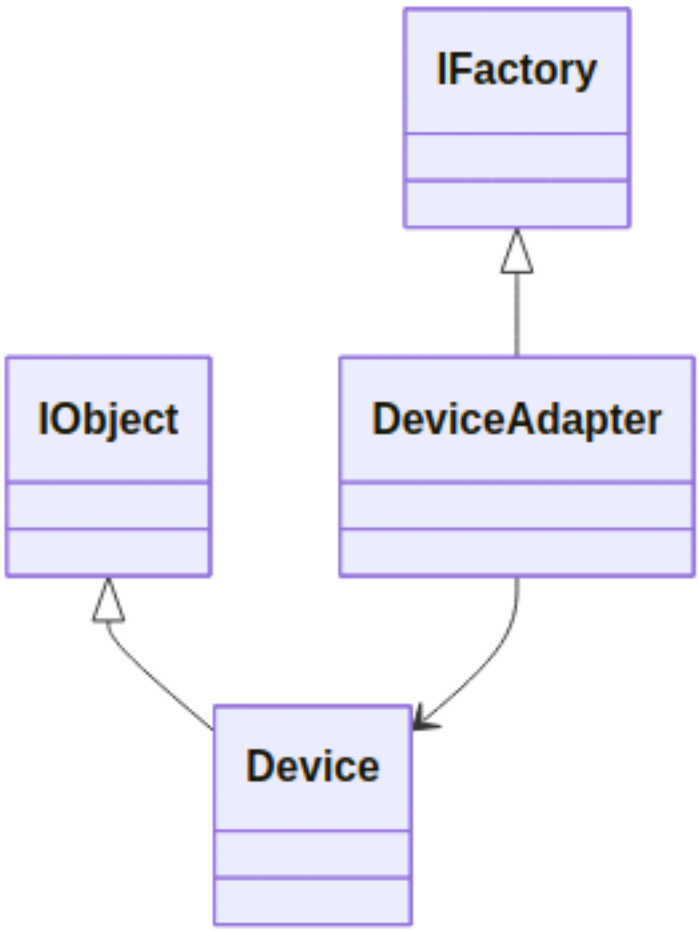

Figure 22: Diagram 22

4.2. External Libraries